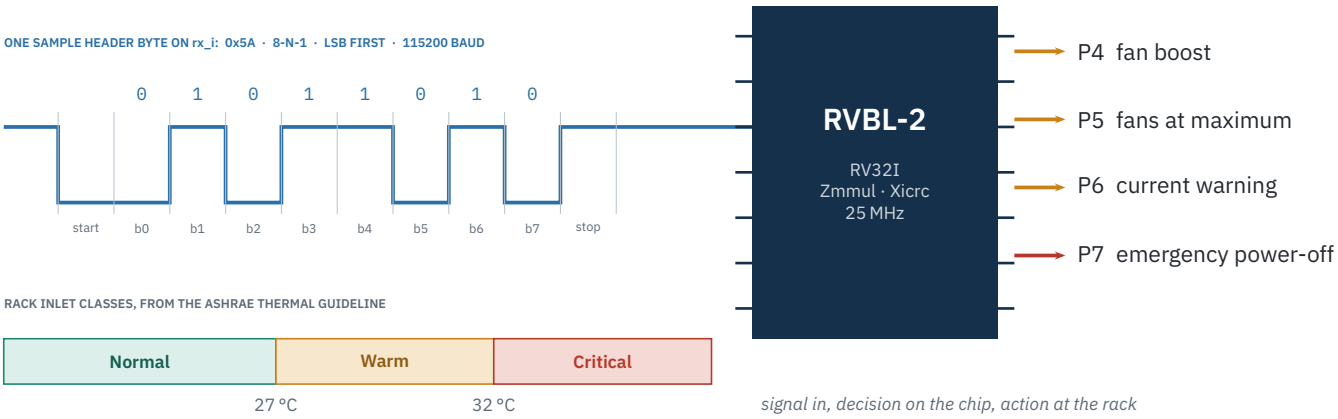
GUIDE TOPICS 1–6 · RVBL-2 RISC-V SOC

# RackSentinel

A data-centre rack thermal and current guardian on the RVBL-2 RISC-V SoC with GPIO and UART




*signal in, decision on the chip, action at the rack*

## KEY RESULTS

**SoC** RVBL-2 core (RV32I + Zmmul + Xicrc) with GPIO (8 pins) and UART (115200 8-N-1), 25 MHz

### 275
**instructions**
`main.s`, 1,100 B, 8 commands; CRC-16 with `crcb`, 0.1 °C with `mul`; latching EPO

### PASS
**official test firmware**
on the ChipInventor platform: 20 checks, 0 failures; GPIO `0xA` to `0xA`, UART `0x30` to `0x30` (§3)

### 124 / 0
**checks / failures**
team testbench `tb_data_center_application.v`, 12 cases, each also run on the platform (§6)

### 12 / 12
**`./emu` runs verified**
each from reset, ≤ 8 reply bytes, ≤ 2.61 ms of simulated time (§4.4, §6)

### 4 / 4
**planted firmware bugs caught**
mutation test of the latching-EPO logic (§6)

### 0 of 18,472
**bit-error patterns missed**
by CRC-16 on the 6-byte packet; an 8-bit XOR checksum misses 80 (§5.2)

*Results from the ChipInventor platform and Icarus Verilog runs of the exported design; §6 lists each test.*

TEAM
**CoreX**
Group 31

MEMBERS
Law Hui Tan
Chia Jing Xiang
Ooi Chin Ming
Lee Wen Yi
Leow Jia Jun

LINKS
Video: YouTube
https://youtu.be/03oN4n_qd6s?si=G52xxw6TRxy75T5m
Source, firmware and testbenches:
Google Drive folder
https://drive.google.com/drive/folders/1wFDAOJwkhgssV14Z5MkH3YdpN-Mq0FtX?usp=drive_link
GitHub repository:
GitHub repo
https://github.com/LemonAidLee/Equipe31_RackSentinel

READING GUIDE

# Contents


1 **Project Overview** . . . 3

2 **GPIO & UART Integration: System Architecture** . . . 3

2.1 The ChipInventor block diagram . . . 4

3 **Official Firmware Testbench** . . . 5

4 **Application Definition** . . . 5

4.1 System context . . . 5

4.2 System input . . . 5

4.3 System processing and output . . . 6

4.4 Emulator values (./emu) . . . 6

5 **Firmware Development and Processing** . . . 6

5.1 UART handling . . . 6

5.2 CRC with the Xicrc instruction . . . 7

5.3 Temperature with Zmmul . . . 7

5.4 GPIO control, latching EPO and thresholds . . . 8

6 **Custom Firmware Validation and Testbench** . . . 8

7 **Conclusion** . . . 8


SYSTEM AT A GLANCE

| | | | |
|---|---|---|---|
| **Core** | RVBL-2 multicycle, RV32I + Zmmul + Xicrc, 25 MHz | **Firmware** | `main.s`: 275 instructions, 1,100 B |
| **UART** | 115200 baud, 8-N-1, divisor 217 | **Packets** | 6-byte SAMPLE (V, I, T); 4-byte commands; CRC-16-CCITT |
| **GPIO** | 8 pins: P7–P4 outputs, P0 reset input, P1–P3 unused | **Thresholds** | Warm ≥ 27 °C, Critical > 32 °C; warning ≥ 25.6 A, trip > 32 A |
| **Memory map** | IMEM `0x0040_0000`, DMEM `0x1001_0000` (8 KB) | **Outputs** | P4 fan boost, P5 fans max, P6 current warning, P7 EPO |
| **Peripherals** | GPIO `0xF000_0000`, UART `0xF100_0000` | **Verification** | 124 checks / 0 failures; 4 of 4 faults detected; 12 `./emu` runs |

WHERE EACH SUBMISSION GUIDE REQUIREMENT IS ANSWERED

| Guide topic | Where it is answered | Evidence |
|---|---|---|
| 1 Project Overview | §1, Figure 1 | Problem, system concept, workflow |
| 2 GPIO & UART Integration | §2, Figures 2–4, Tables 1–2 | Integration design; ChipInventor block diagram (mandatory) |
| 3 Official Firmware Testbench | §3, Figure 5 | Log and waveform of the organizer firmware on the exported design |
| 4 Application Definition | §4, Tables 3–5 | Context, input, processing, output, `./emu` values |
| 5 Firmware Development | §5, Figure 6 | UART handling, CRC, Zmmul, GPIO control, latching EPO |
| 6, 7, 9 Validation, video, files | §6, Table 6, Figures 7–8; cover | Test summary; links; binary + assembly |

FIGURES


1 System concept: inputs, on-chip decision, outputs . . . 3

2 SoC integration architecture . . . 4

3 Block diagram generated in ChipInventor (mandatory) . . . 4

4 Close-up of the peripheral side of the canvas . . . 5

5 Official firmware testbench: platform log and waveform . . . 5

6 Firmware decision flow . . . 7

7 Validation matrix of the 12 ./emu runs . . . 9

8 Application testbench: waveform and platform log of case C11 . . . 9


TABLES


1 Memory map and peripheral registers . . . 3

2 Integration design decisions . . . 4

3 Command set . . . 6

4 Decision table: conditions, triggers and pins . . . 6

5 Verified ./emu demonstration runs . . . 7

6 Verification summary . . . 8

# 1 Project Overview

GUIDE TOPIC 1

OBJECTIVE Protect one data-centre rack against overheating and feed overload with an on-chip controller.
RESULT **RVBL-2 SoC with GPIO and UART; 275-instruction firmware** EVIDENCE Figure 1; summary on page 2

A cooling failure or an overloaded power feed can take a server rack offline. RackSentinel places the protection decision on the RVBL-2 SoC that guards one server rack in a Johor data hall. The data-centre management system (DCIM) forwards the rack's inlet temperature, feed voltage and feed current to the chip over the UART, together with configuration and status commands. The firmware rejects corrupted packets with the custom CRC-16 instruction `crcb`, scales the temperature with the Zmmul `mul` instruction and classifies both readings. It then drives four GPIO outputs: fan boost, fans at maximum, a current warning and an **emergency power-off (EPO)** that stays latched until an operator resets it on P0 under safe conditions (Figure 1).

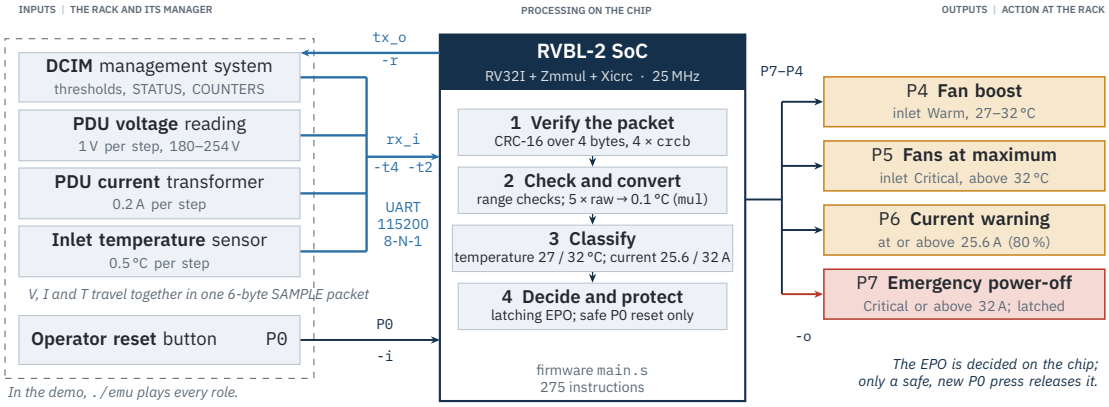
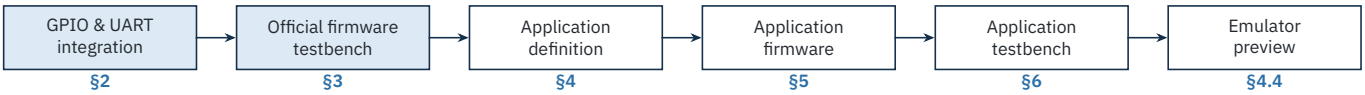



**Figure 1** System concept. Readings and commands arrive on the UART (blue) and the reset button on P0 (navy); the chip verifies, converts, classifies and drives four outputs. Monospaced labels give the `./emu` option for each connection.

The Stage 2 core is reused unchanged; Stage 3 adds the GPIO and UART peripherals and modifies only the address decoder (§2). In the demonstration the organizer's emulator, `./emu`, replaces the DCIM and the reset button, as a hardware-in-the-loop bench would. All results come from simulation of the design exported by ChipInventor, on the ChipInventor platform and in Icarus Verilog; direct AWS FPGA implementation is outside this phase [1]. The work followed the Phase 3 workflow in order:




# 2 GPIO & UART Integration: System Architecture

GUIDE TOPIC 2

OBJECTIVE Connect GPIO and UART to the Stage 2 core and enter the design in ChipInventor.
RESULT **One decoder change; 16 blocks and 8 Inout Pins on the canvas** EVIDENCE Figures 2–4, Tables 1–2

GPIO and UART are bus slaves behind the address decoder, the only core block modified in Stage 3 (Figure 2). The decoder selects a slave from the upper address bits, forwards the bus signals and returns the selected read data to the core. Each peripheral decodes only its word offset, address bits 11:2, as the Block Guide requires [2]. The firmware therefore reaches the pins and the serial line with ordinary `lw` and `sw` instructions (Table 1); Table 2 records the design decisions.

**Table 1** Memory map and peripheral registers, from the Block Guide Stage 3 register maps [2] and `Addr_dec.v`.

| Block | Base address | Registers (offset) / contents |
|---|---|---|
| IMEM (ROM) | `0x0040_0000` | `IMEM_Data_Center` (275 words) or the official test firmware (21 words) |
| DMEM | `0x1001_0000` | 8 KB SRAM (2,048 words); application state in 22 words, cleared at boot |
| GPIO | `0xF000_0000` | `DATAOUT` (`0x0`), `DATAIN` (`0x4`), `DATADIR` (`0x8`; 1 = output) |
| GPIO_PIN_SPLIT | none (wiring) | none; per pin C = `DATAOUT`[$k$], D = `DATADIR`[$k$], P = pin |
| UART | `0xF100_0000` | `TXDATA` (`0x0`), `RXDATA` (`0x4`), `CONTROL` (`0x8`): `TRANSMIT` bit 0, `RXDONE` bit 1, `TXDONE` bit 2 |

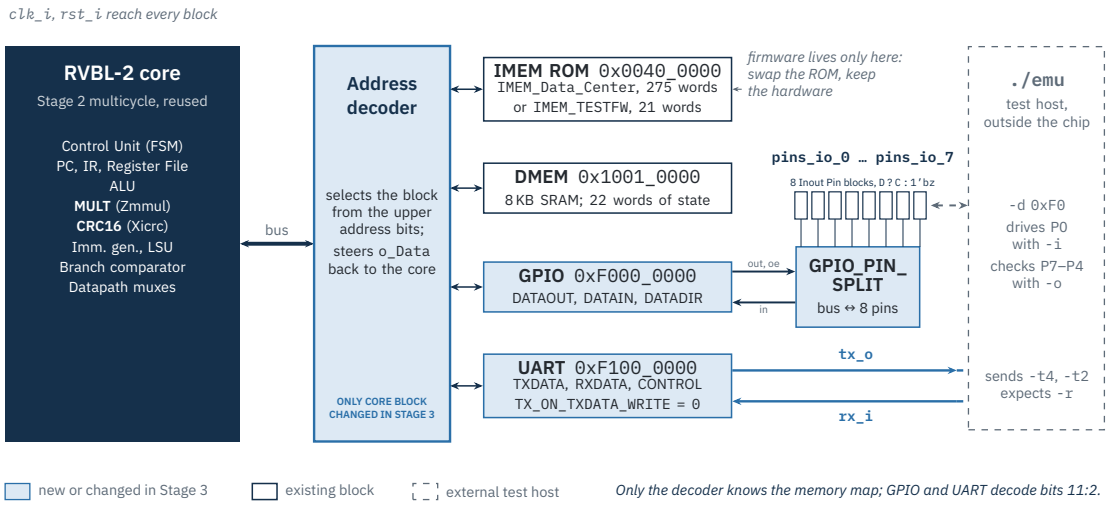



**Figure 2** SoC integration. The address decoder is the only modified core block; GPIO and UART are new bus slaves, and each pin has its own Inout Pin block. The two firmwares differ only in the ROM.

**Table 2** Integration design decisions and their rationale.

| Area | Decision | Rationale |
|---|---|---|
| GPIO reset | All pins are inputs; DATAOUT = 0 | No pin is driven before the firmware writes DATADIR. |
| GPIO input | 2-flop synchronizer before DATAIN; in output mode a pin's DATAIN bit holds its last input value | Pins are asynchronous to the clock; the Guide isolates the input buffer of an output pin (§1.3.1) [2]. |
| Pin tristate | GPIO_PIN_SPLIT feeds eight 1-bit Inout Pin blocks: `pins_io_k = D ? C : 1'bz` | An 8-bit Inout Pin applies one enable to all pins, and the export drops a custom block's `inout` port; scalar pins avoid both without editing `hdl.v`. |
| UART baud | $N = 25\,\text{MHz}/115\,200 = 217$ clocks per bit | Integer divisor of the system clock; 0.01 % baud error. |
| UART transmit | A frame starts only when TRANSMIT is set (`TX_ON_TXDATA_WRITE = 0`); TRANSMIT self-clears; TXDONE = 1 when idle | Specified behaviour: writing TXDATA only loads the byte. |
| UART receive | Synchronized input; start bit re-checked at half a bit; one sample per bit centre; frame dropped if the stop bit is low; RXDONE cleared by writing 0, and a byte arriving in the same cycle takes priority | Rejects glitches and framing errors without oversampling; no received byte is lost at the clear. |

## 2.1 The ChipInventor block diagram

Figure 3 is the final design as entered on the canvas, with one wire colour per bus; the numbered markers identify the blocks. The canvas exports `hdl.v`, submitted as `rtl/hdl/hdl_application.v` and used by every test in §6 without modification. Each custom block is identical to the repository RTL.

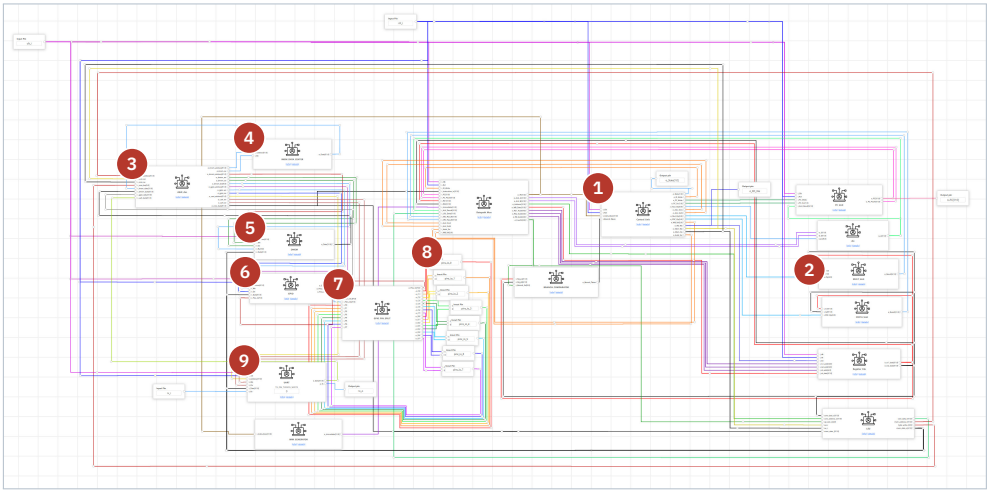



**Figure 3** Block diagram generated in ChipInventor (mandatory).

1. Control Unit
2. PC, ALU, MULT, CRC16, Register File, LSU
3. Address decoder
4. IMEM ROM, 275 words
5. DMEM, 8 KB
6. GPIO
7. GPIO_PIN_SPLIT
8. Inout Pins `pins_io_0–7`
9. UART

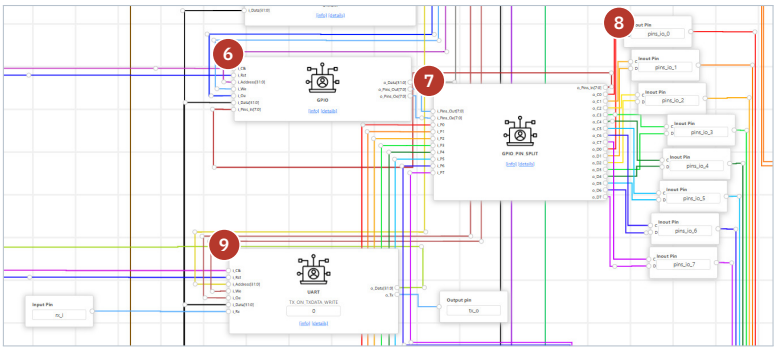



**Figure 4** Close-up of Figure 3. GPIO 6 sends `o_Pins_Out` and `o_Pins_Oe` to `GPIO_PIN_SPLIT` 7, which drives C and D of each Inout Pin 8 and returns each pin on `i_P0`–`i_P7`. The UART 9 connects to `rx_i` and `tx_o` with `TX_ON_TXDATA_WRITE = 0`.

# 3 Official Firmware Testbench

GUIDE TOPIC 3

OBJECTIVE Validate the GPIO and UART integration with the organizer's firmware before any application code.
RESULT **PASS: 20 checks, 0 failures on the platform** EVIDENCE Figure 5

The organizer's **RVBL-GPIO-UART-Test-Firmware** [3] configures P7–P4 as outputs, copies P3–P0 to P7–P4, then waits for one UART byte and echoes it. For this test the canvas carries `IMEM_TESTFW` instead of the application ROM: a 1,024-word ROM whose first 21 words match the organizer's `firmware.txt` word for word and whose other words are `nop`. The rest of the exported `hdl.v` is identical to the application design. The official testbench (`tb_official_test.v`) drives only the top-level pins and puts a timeout on every wait, plus a global watchdog:

1. Check that the firmware sets `DATADIR` to `0xF0` (P7–P4 outputs).
2. Official GPIO test: `0xA` on P3–P0 before reset is released; expect `0xA` on P7–P4.
3. Official UART test: send `0x30` on `rx_i`; expect `0x30` on `tx_o` with valid start and stop bits.
4. Ordering: change P3–P0 to `0x5`; P7–P4 must hold `0xA` until the `0x55` echo, then show `0x5`.
5. Four further patterns (GPIO, UART): (`0x2`, `0x20`), (`0xF`, `0xA5`), (`0x9`, `0x00`), (`0x6`, `0xFF`).

On the ChipInventor platform all 20 checks pass and the run ends at 1.062 ms (26,551 cycles at 25 MHz); Icarus Verilog 12 gives the same result. The equivalent emulator command is `./emu -d 0xF0 -i 0x0A -t1 0x30 -r1 0x30 -o 0xA0`.

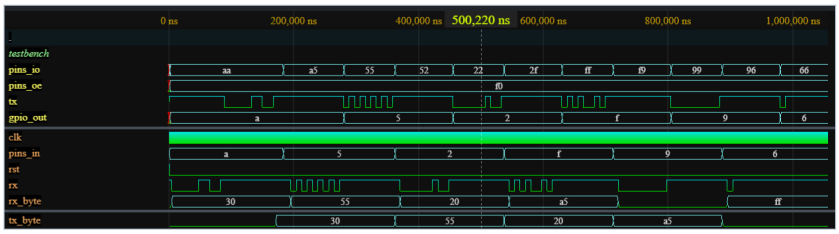
Simulation Log

```
VCD info: dumpfile testbench/testbench.vcd opened for output.
============================================================
 Official Stage 3 GPIO/UART Firmware Test
============================================================
[PASS] GPIO: P3-P0 = 0xA, P7-P4 = 0xA
[PASS] UART: RX = 0x30, TX = 0x30
Equivalent ./emu: -d 0xF0 -i 0x0A -t1 0x30 -r1 0x30 -o 0xA0
[PASS] Firmware ordering: GPIO update occurs after UART echo

[ADDITIONAL VERIFICATION]
[PASS] Test 1: GPIO 0x2 -> 0x2 | UART 0x20 -> 0x20
[PASS] Test 2: GPIO 0xf -> 0xf | UART 0xa5 -> 0xa5
[PASS] Test 3: GPIO 0x9 -> 0x9 | UART 0x00 -> 0x00
[PASS] Test 4: GPIO 0x6 -> 0x6 | UART 0xff -> 0xff

Internal checks: 20, failures: 0
[PASS] GPIO and UART firmware test completed successfully
============================================================
testbench/testbench.v:396: $finish called at 1062040000 (1ps)
```

**(a)** Simulation log on the ChipInventor platform.




**(b)** Waveform of the same run in the platform viewer (0–1.05 ms; testbench parameter rows hidden). `gpio_out` (P7–P4) follows `pins_in` (P3–P0) only after each echo; `tx_byte` repeats `rx_byte`.

**Figure 5** Official firmware on the exported design, on the ChipInventor platform. All 20 checks pass, and each new P3–P0 value reaches P7–P4 only after the next UART echo, as the firmware README specifies.

# 4 Application Definition

GUIDE TOPIC 4

OBJECTIVE Define the input, processing and output of the rack controller and its emulator interface.
RESULT **8 commands, 7 decision rules, 12 emulator runs** EVIDENCE Tables 3–5

## 4.1 System context

The controller protects one server rack in a Johor data hall, one of the fastest-growing data-centre locations in the region. The temperature thresholds follow the ASHRAE thermal guideline for data-processing environments [4]: the recommended inlet range ends at **27 °C** (Warm from this value) and the class A1 allowable maximum is **32 °C** (Critical above it). The rack is fed by a 32 A, 230 V circuit. With the usual limit of 80 % of a breaker's rating for a continuous load, the warning level is $I_{\text{warn}} = 0.8 \times 32\ \text{A} = \mathbf{25.6\ A}$, and a current above the 32 A rating trips the EPO.

## 4.2 System input

Readings arrive as one 6-byte **SAMPLE** packet: header `0x5A`, voltage in 1 V steps, current in 0.2 A steps and temperature in 0.5 °C steps, then the CRC (example below). Configuration and status use 4-byte commands, `[CMD] [DATA] [CRC_LO]`

`[CRC_HI]` (Table 3). Both formats end with a CRC-16-CCITT (polynomial `0x1021`, seed `0xFFFF`, the CCSDS frame check [5]) over the preceding bytes. P0 is the operator reset button; P1–P3 are not used.




PACKET ANATOMY: CASE C03, 230 V · 25.4 A · 32.5 °C

CRC-16 of the first four bytes = 0xCB43, low byte first

`-t4 0x417FE65A -t2 0xCB43`

the same packet as `./emu` options (§4.4)

**Table 3** Command set. Replies: `0xA5` accepted, `0x5A` bad CRC (nothing executed), `0x3C` invalid or unknown.

| CMD | Name | Data | Reply | Effect |
|---|---|---|---|---|
| `0x5A` | SAMPLE | V 180–254; I 0–254; T 1–254 | A5 / 3C | Store V and I; T × 5 (`mul`) → 0.1 °C; classify; current warning; EPO trip |
| `0x40` | STAGE_HIGH | high byte | A5 | Stage the high byte of a temperature threshold |
| `0x41` / `0x43` | SET_WARM / SET_CRIT | low byte | A5 / 3C | Commit (0.1 °C units) only after STAGE_HIGH and only if Warm < Critical |
| `0x46` / `0x48` | SET_WARN_CURRENT / SET_TRIP_CURRENT | raw 1 to trip—1 / above warning, not 255 | A5 / 3C | Current warning level (default 128 = 25.6 A) / trip level (default 160 = 32.0 A) |
| `0x50` | STATUS | 0 | 4 bytes | Flags (class, warning, EPO, temperature received, staged), temperature, current, EPO trips |
| `0x52` | COUNTERS | 0 | 4 bytes | CRC rejects, then three zero bytes |
| other | any other value | — | 3C | Unknown command: nothing executed |

### 4.3 System processing and output

Table 4 is the complete rule set. A packet with a bad CRC or out-of-range data receives its reply (`0x5A` or `0x3C`) and changes no state. Classification uses the latest SAMPLE; a reading of exactly 32.0 °C remains Warm, and exactly 32.0 A remains a warning. Each complete packet receives one reply.

**Table 4** Decision table: each condition the firmware evaluates, its trigger and basis, the pins it drives and the value `./emu -o` reads. Pin values add, e.g. `0xE0` = fans max + warning + EPO.

| Condition | Trigger and basis | Pins driven | `-o` |
|---|---|---|---|
| Normal | inlet below 27.0 °C, inside the ASHRAE recommended range | none | `0x00` |
| Warm | 27.0 °C ≤ inlet ≤ 32.0 °C | P4 fan boost | `0x10` |
| Critical temperature | inlet above 32.0 °C, the class A1 allowable maximum | P5 fans at maximum + P7 EPO | `0xA0` |
| Current warning | I ≥ 25.6 A (80 % of the 32 A feed); follows the latest reading | P6 current warning | `0x40` |
| Current trip | I above 32.0 A, the feed rating | P6 warning + P7 EPO | `0xC0` |
| EPO latched | stays on after its cause is gone, until a valid reset | P7 EPO (+ current P4–P6) | `0x80` |
| EPO reset | new P0 press while the inlet is Normal and I < 25.6 A | releases P7; a held button never does | — |

### 4.4 Emulator values (./emu)

The organizer's emulator [3] drives the pins and the UART of the FPGA top. Each run resets the processor and stores at most 8 reply bytes. Every register access waits 50 ms, so consecutive UART bytes reach the chip at least 100 ms apart. Table 5 lists the 12 demonstration runs, which use the options on the right.

| | |
|---|---|
| `-d 0xF0` | P7–P4 observed by the emulator, P3–P0 driven by it |
| `-i V` | Drive P3–P0: `0x01` presses P0, `0x00` releases it |
| `-o V` | Compare P7–P4 with V |
| `-t4`, `-t2` | Send 4 or 2 bytes, least-significant first: SAMPLE `-t4 0x{T}{I}{V}5A -t2 0x{CRC}`; command `-t4 0x{CRC}{DATA}{CMD}` |
| `-rN V` | Expect N reply bytes; the first byte is the least significant |

## 5 Firmware Development and Processing

GUIDE TOPIC 5

OBJECTIVE Implement the application in RV32I assembly using the custom CRC and Zmmul instructions.

RESULT **275 instructions; rebuild matches the ROM byte for byte** EVIDENCE Figure 6; listings in §5.2 and §5.4

The firmware is a single RV32I assembly file, `main.s`: 275 instructions, 1,100 B. The organizer's RVBL-Firmware-Builder [3] (GNU toolchain, `-march=rv32i_zmmul`) rebuilds it into exactly the submitted `firmware.bin`, whose 275 words match `IMEM_Data_Center` in the exported design; `crcb` is emitted through a `.insn` macro because the assembler does not know the custom opcode. Application state occupies 22 words of DMEM at `0x1001_0000` and is cleared at boot. Figure 6 traces one packet from arrival to reply.

### 5.1 UART handling

`rx_byte_wait` polls `CONTROL.RXDONE` for the first byte without a time limit and calls `check_reset` between polls, so the reset button is served while the line is idle. `rx_byte_timeout` then reads the remaining 5 bytes of a SAMPLE or 3 bytes of a command. Each byte has a limit of 12,500,992 polls of 20 cycles (measured): 10 s at 25 MHz or 1.0 s at 250 MHz, well

**Table 5** Emulator command lines, printed by the testbench after each case passed; Figure 7 decodes each step. Example: C03 STATUS 0x017F414B = flags 0x4B (Critical, EPO, temperature received), 32.5 °C (0x41), 25.4 A (0x7F), 1 EPO trip.

| Case | ./emu command line (verified) |
|---|---|
| C01 | `./emu -d 0xF0 -t4 0x357FE65A -t2 0xF550 -r1 0xA5 -o 0x00 -t4 0x13B00050 -r4 0x007F3540` |
| C02 | `./emu -d 0xF0 -t4 0x367FE65A -t2 0xC533 -r1 0xA5 -o 0x10 -t4 0x407FE65A -t2 0xDB62 -r1 0xA5 -o 0x10 -t4 0x13B00050 -r4 0x007F4041` |
| C03 | `./emu -d 0xF0 -t4 0x417FE65A -t2 0xCB43 -r1 0xA5 -o 0xA0 -t4 0x13B00050 -r4 0x017F414B` |
| C04 | `./emu -d 0xF0 -t4 0x327FE65A -t2 0x85B7 -r1 0xA5 -o 0x00 -t4 0x3280E65A -t2 0x8648 -r1 0xA5 -o 0x40 -t4 0x32A0E65A -t2 0x80AE -r1 0xA5 -o 0x40` |
| C05 | `./emu -d 0xF0 -t4 0x32A1E65A -t2 0xB39F -r1 0xA5 -o 0xC0 -t4 0x13B00050 -r4 0x01A1324C` |
| C06 | `./emu -d 0xF0 -t4 0x41A1E65A -t2 0xFD6B -r1 0xA5 -o 0xE0 -t4 0x32A1E65A -t2 0xB39F -r1 0xA5 -o 0xC0 -t4 0x417FE65A -t2 0xCB43 -r1 0xA5 -o 0xA0` |
| C07 | `./emu -d 0xF0 -t4 0x41A1E65A -t2 0xFD6B -r1 0xA5 -o 0xE0 -t4 0x3680E65A -t2 0xC6CC -r1 0xA5 -o 0xD0 -t4 0x357FE65A -t2 0xF550 -r1 0xA5 -o 0x80 -i 0x01 -o 0x00 -i 0x00` |
| C08 | `./emu -d 0xF0 -t4 0x417FE65A -t2 0xCB43 -r1 0xA5 -o 0xA0 -t4 0x407FE65A -t2 0xDB62 -r1 0xA5 -o 0x90 -t4 0x357FE65A -t2 0xF550 -r1 0xA5 -o 0x80 -i 0x01 -o 0x00 -i 0x00` |
| C09 | `./emu -d 0xF0 -t4 0x32A1E65A -t2 0xB39F -r1 0xA5 -o 0xC0 -t4 0x3280E65A -t2 0x8648 -r1 0xA5 -o 0xC0 -i 0x01 -o 0xC0 -i 0x00 -t4 0x327FE65A -t2 0x85B7 -r1 0xA5 -o 0x80 -i 0x01 -o 0x00 -i 0x00` |
| C10 | `./emu -d 0xF0 -t4 0x417FE65A -t2 0xCB43 -r1 0xA5 -o 0xA0 -t4 0x007FE65A -t2 0x93A6 -r1 0x3C -o 0xA0 -t4 0x357FE65A -t2 0xF551 -r1 0x5A -o 0xA0` |
| C11 | `./emu -d 0xF0 -t4 0x417FE65A -t2 0xCB43 -r1 0xA5 -o 0xA0 -i 0x01 -o 0xA0 -i 0x00 -t4 0x367FE65A -t2 0xC533 -r1 0xA5 -o 0x90 -i 0x01 -o 0x90 -i 0x00 -t4 0x357FE65A -t2 0xF550 -r1 0xA5 -o 0x80 -i 0x01 -o 0x00 -i 0x00` |
| C12 | `./emu -d 0xF0 -i 0x01 -t4 0x32A1E65A -t2 0xB39F -r1 0xA5 -o 0xC0 -t4 0x327FE65A -t2 0x85B7 -r1 0xA5 -o 0x80 -i 0x00 -i 0x01 -o 0x00 -i 0x00` |

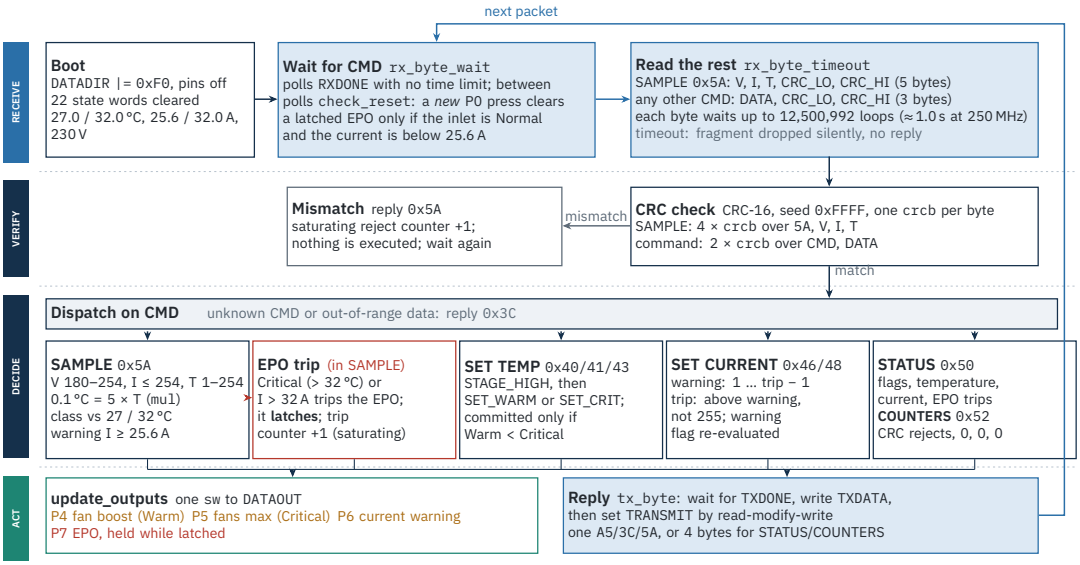



**Figure 6** Firmware control flow in four lanes. A complete packet produces one reply; a timed-out fragment produces none.

above the 100 ms emulator gap. On a timeout the fragment is dropped without a reply, so a lost byte cannot misalign later packets. Each received byte is acknowledged by writing 0 to RXDONE before RXDATA is read; `tx_byte` waits for TXDONE, writes TXDATA and sets TRANSMIT by read-modify-write, as the organizer's firmware does.

## 5.2 CRC with the Xicrc instruction

The firmware assembles the received CRC from CRC_LO and CRC_HI (`slli`, `or`) and recomputes it from `0xFFFF` with one `crcb` per byte: four for a SAMPLE, two for a command. A mismatch returns `0x5A`, increments a saturating reject counter (read with COUNTERS) and executes nothing. The algorithm reproduces the CRC-16-CCITT check value `0x29B1` for "123456789".

```
li   t0, 0xFFFF
crcb t0, s2, t0          # s2 = 0x5A
crcb t0, s3, t0          # s3 = V
crcb t0, s4, t0          # s4 = I
crcb t0, s5, t0          # s5 = T
bne  t0, t5, cmd_badcrc  # received CRC
```

**VERIFIED: CRC AGAINST A CHECKSUM**

All 1-, 2- and 3-bit errors in a SAMPLE packet (18,472 patterns): the CRC misses **none**; an 8-bit XOR checksum misses **80** of 10,700.

## 5.3 Temperature with Zmmul

The single multiplication converts a temperature reading from 0.5 °C to 0.1 °C steps, $T_{0.1\,^\circ\mathrm{C}} = 5 \times raw_T$ (`mul`). Signed comparisons with the Warm and Critical thresholds (defaults 270 and 320) give class 0 (Normal), 1 (Warm) or 3 (Critical). Current is compared in raw 0.2 A steps with 128 (25.6 A) and 160 (32.0 A), so the firmware contains no division.

### 5.4 GPIO control, latching EPO and thresholds

`update_outputs` writes the pin word to DATAOUT with one sw: P4 for Warm, P5 for Critical, P6 for a current warning and P7 while the EPO is latched. A SAMPLE that is Critical or above the trip current sets the EPO and increments a saturating trip counter; later safe readings leave it set. Only `check_reset` (right) clears it: on a new press of P0, with the inlet Normal and the current below the warning level. A button held through a trip therefore never clears it.

Temperature thresholds change through STAGE_HIGH followed by SET_WARM or SET_CRIT and are committed only if Warm < Critical; current levels must keep warning < trip.

```
check_reset:                    # excerpt
  lw   a4, GPIO_DATAIN(s0)
  andi a4, a4, PIN_RESET        # P0 level
  lw   t1, ST_BTN_PREV(s11)
  sw   a4, ST_BTN_PREV(s11)
  beqz a4, _cr_ret              # not pressed
  bnez t1, _cr_ret              # held: no new press
  lw   t0, ST_EPO(s11)
  beqz t0, _cr_ret              # EPO not set
  lw   t0, ST_CLASS(s11)
  bnez t0, _cr_ret              # inlet not Normal
  lw   t0, ST_LATEST_CURRENT(s11)
  lw   t1, ST_WARN_CURRENT(s11)
  bgeu t0, t1, _cr_ret          # I >= warning level
  sw   zero, ST_EPO(s11)        # release the EPO
  jal  ra, update_outputs
```

# 6 Custom Firmware Validation and Testbench

GUIDE TOPIC 6

OBJECTIVE Show that the firmware meets Table 4 and that the testbench detects faults.
RESULT **124 checks, 0 failures; 4 of 4 faults detected** EVIDENCE Table 6, Figures 7–8

The application testbench `tb_data_center_application.v` drives only the chip pins, as `./emu` does, and checks every reply byte and pin value; expected CRCs come from an independent bit-serial model. Figure 7 shows its 12 cases. Because the platform stops a simulation after about 3 ms, each case was run on its own on the ChipInventor platform (FIRST_RUN = LAST_RUN = $k$); the 12 platform logs add up to 124 checks with 0 failures, and a single Icarus Verilog run of all 12 cases gives the same totals. Table 6 summarizes the evidence.

**Table 6** Verification summary. Every row can fail; section 11 of the package README gives the command that reproduces it.

| Test | Scope | Result | Evidence |
|---|---|---|---|
| Official firmware `tb_official_test.v` | DATADIR, GPIO copy, UART echo, ordering, four further patterns | **PASS**: 20 checks, 0 failures (platform and Icarus) | §3, Figure 5 |
| Application testbench `tb_data_center_application.v` | 12 cases: boundaries, single faults, latch and reset rules, held button, invalid data, bad CRC | **PASS**: 124 checks, 0 failures; each case also alone on the platform | Figures 7, 8 |
| Emulator runs `emu_runs.txt`, `emu.sh` | Each case from reset, ≤ 8 reply bytes, ≤ 2.61 ms simulated; the 12 lines replayed with the `emu.c` option semantics | **PASS**: 12 of 12 within limits; replay 12 of 12 | Table 5 |
| Mutation test `mutate_final.sh` | 4 bugs in the latch logic: auto-clear restored, reset while Warm, reset at warning current, level-triggered reset | **PASS**: 4 of 4 detected (5, 4, 2, 1 failing checks) | this table |
| CRC strength `crc_verify.py` | Check value `0x29B1`; all 1-, 2- and 3-bit errors in a SAMPLE packet | **PASS**: 0 of 18,472 missed (XOR checksum: 80 of 10,700) | §5.2 |
| Firmware rebuild organizer `Makefile` | `main.s` → `firmware.bin` → `IMEM_Data_Center` | **PASS**: byte-identical, 275 of 275 words | §5 |

# 7 Conclusion

RESULTS AND OUTLOOK

RackSentinel integrates GPIO and UART with the reused RVBL-2 core through one decoder change and eight native Inout Pin blocks, and the organizer's firmware passes on the exported design. The 275-instruction application verifies each packet with `crcb`, scales the temperature with `mul`, and holds the EPO until a new P0 press arrives under safe conditions. The design passes 124 of 124 checks across 12 cases, each run on the ChipInventor platform and each available as an `./emu` run for the FPGA stage, and the testbench detects all four faults planted in the latch logic.

## References

[1] ChampionCHIP eXperience. Phase 3 Challenge: Design Submission Guide, 2026. ChampionCHIP eXperience Malaysia.

[2] ChampionChip. Block Guide, Stage 3: PIN Controller (GPIO) and Serial Controller (UART), 2026. Version 1.0.

[3] ChampionCHIP eXperience Community. CCX_Malaysia_Edition_Stage_3: RVBL-GPIO-UART-Test-Firmware, RVBL-Emulator and RVBL-Firmware-Builder. https://github.com/championchip-experience-community/CCX_Malaysia_Edition_Stage_3.

[4] ASHRAE Technical Committee 9.9. Thermal Guidelines for Data Processing Environments. ASHRAE.

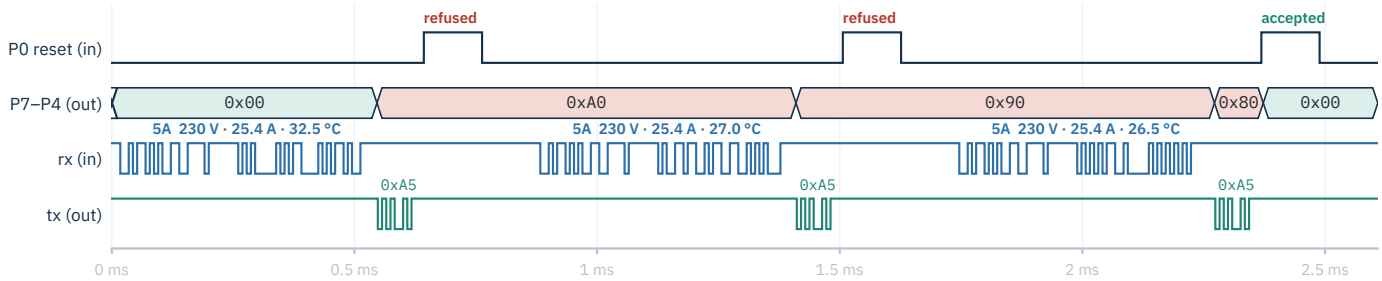
[5] Consultative Committee for Space Data Systems. TC Space Data Link Protocol. CCSDS 232.0-B (frame error control field, CRC-16).

| CASE AND SCENARIO | | STIMULUS FROM ./emu → REPLY (-r) · EXPECTED PINS P7–P4 (-o), IN ORDER |
|---|---|---|
| **C01** | Normal temperature and current | 25.4 A · 26.5 °C `A5 · 00` → STATUS `007F3540` |
| **C02** | Warm boundaries: 27.0 and 32.0 °C | 25.4 A · 27.0 °C `A5 · 10` → 25.4 A · 32.0 °C `A5 · 10` → STATUS `007F4041` |
| **C03** | Temperature Critical | 25.4 A · 32.5 °C `A5 · A0` → STATUS `017F414B` |
| **C04** | Current boundaries: 25.6 and 32.0 A | 25.4 A · 25.0 °C `A5 · 00` → 25.6 A · 25.0 °C `A5 · 40` → 32.0 A · 25.0 °C `A5 · 40` |
| **C05** | Current Critical (trip) | 32.2 A · 25.0 °C `A5 · C0` → STATUS `01A1324C` |
| **C06** | Either fault is enough | 32.2 A · 32.5 °C `A5 · E0` → 32.2 A · 25.0 °C `A5 · C0` → 25.4 A · 32.5 °C `A5 · A0` |
| **C07** | Both must become Normal | 32.2 A · 32.5 °C `A5 · E0` → 25.6 A · 27.0 °C `A5 · D0` → 25.4 A · 26.5 °C `A5 · 80` → P0 pressed `00 cleared` |
| **C08** | Warm temperature blocks recovery | 25.4 A · 32.5 °C `A5 · A0` → 25.4 A · 32.0 °C `A5 · 90` → 25.4 A · 26.5 °C `A5 · 80` → P0 pressed `00 cleared` |
| **C09** | Warning current blocks recovery | 32.2 A · 25.0 °C `A5 · C0` → 25.6 A · 25.0 °C `A5 · C0` → P0 pressed `C0 refused` → 25.4 A · 25.0 °C `A5 · 80` → P0 pressed `00 cleared` |
| **C10** | Invalid packets change nothing | 25.4 A · 32.5 °C `A5 · A0` → temperature raw 0 `3C · A0` → corrupted CRC `5A · A0` |
| **C11** | P0 refused until temperature and current are safe | 25.4 A · 32.5 °C `A5 · A0` → P0 pressed `A0 refused` → 25.4 A · 27.0 °C `A5 · 90` → P0 pressed `90 refused` → 25.4 A · 26.5 °C `A5 · 80` → P0 pressed `00 cleared` |
| **C12** | A held P0 never clears the EPO | P0 held down `–` → 32.2 A · 25.0 °C `A5 · C0` → 25.4 A · 25.0 °C `A5 · 80` → release, press again `00 cleared` |

`00` all off / EPO cleared · 10 P4 fan boost, 40 P6 current warning · 80 P7 EPO (+ 20 fans max, 10, 40) · reply bytes (`-r`)

*Samples at 230 V unless stated. A5 accepted, 3C invalid, 5A bad CRC; each P0 press is followed by a release (`-i 0x00`).*

**Figure 7** Expected behaviour of the 12 emulator runs, decoded from the verified command lines of Table 5. Each cell is one stimulus and the reply or pin value required at that point; colour follows the rack state.




**(a)** Waveform of case C11 on the exported design, drawn from the VCD of the 12-case run.

**Simulation Log**

```
VCD info: dumpfile testbench/testbench.vcd opened for output.
START C11 P0 reset refused until temperature and current are safe
SAMPLE START: 32.5 C -> Fan Max + EPO ON
SAMPLE DONE: 32.5 C -> Fan Max + EPO ON; reply=a5 outputs=a0 expected=a0
SAMPLE START: 27 C (Warm) -> EPO latched
SAMPLE DONE: 27 C (Warm) -> EPO latched; reply=a5 outputs=90 expected=90
SAMPLE START: 26.5 C, 25.4 A -> safe, EPO latched
SAMPLE DONE: 26.5 C, 25.4 A -> safe, EPO latched; reply=a5 outputs=80 expected=80
CASE TIME C11 P0 reset refused until temperature and current are safe: 2608700 ns
EMU RUN C11 P0 reset refused until temperature and current are safe -- verified, 3 of 8 reply
bytes:
  ./emu -d 0xF0 -t4 0x417FE65A -t2 0xCB43 -r1 0xA5 -o 0xA0 -i 0x01 -o 0xA0 -i 0x00 -t4
0x367FE65A -t2 0xC533 -r1 0xA5 -o 0x90 -i 0x01 -o 0x90 -i 0x00 -t4 0x357FE65A -t2 0xF550 -r1
0xA5 -o 0x80 -i 0x01 -o 0x00 -i 0x00
tb_dc_teammate: 1 cases, 14 checks, 0 failures
tb_dc_teammate PASSED
testbench/testbench.v:391: $finish called at 2608700000 (1ps)
```

**(b)** Simulation log of case C11 alone on the ChipInventor platform.

**Figure 8** Case C11. Two P0 presses are refused (Critical, then Warm); the third, after a safe reading, clears the EPO (P7–P4 `0x80` → `0x00`). On the platform the case passes 14 of 14 checks and prints its verified `./emu` line.